

Problem 1

A) In normal logging in programs, there are attempt rate limits, which would become useless should the hashes are not protected.

B) I suppose there is no real gain in limiting which hash function to use for all users' passwords, except that it will make the system design simpler. In contrast, if the hash functions are unlimited, the system can be easily upgraded in the future with safer hash algorithms (without the user's even knowing about it!)

C) Making a hash evaluation much slower, thus limiting brute force attacks.

D) I would set a reasonable count that can make it harder at least 1000 times for an attacker, while not enough to cause big slow down to the server (ideally, maybe 100th of a second for one authentication), since the server is popular. If 100 people are using the server together, that will take a total of 1 second of CPU time, which is big already.

E) I will use a different strategy on my personal laptop, because only me will be using it, and I don't care if I will wait for a few seconds for one mistyped password, I will set the count so that it will take at least seconds to calculate such a hash.

Problem 2

B) Number of total non-empty strings $\Sigma^{\leq n}$ is 26^{20} . By the birthday paradox, the number of average tries is 26^{20} .

D) Since each salt is unique, the cost would change to be as if we only have a single hash target at a time (ignoring collisions). That is, it would be slowed by the factor of the number of hashes in `hashes.txt`.

Problem 4

A) The problem is not very clearly described. Here is one possible meaning:

One possible meaning Let x be any input. I am given $H_0 := H(x)$, $H_n := H(H_{n-1})$, and I know that for some m , $H_m = H_0$ (assuming no subcycles, i.e., m is the smallest number to have a cycle from H_0).

However, I am not given x ; I only have the values $(H_n)_{n=0}^m$. Because I don't know x , I can't use the collision $H(x) = H(H_{m-1})$; I don't even know if $x = H_{m-1}$ or not.

Note that I have both $H_0 = H_m$ by cycle and $H^m(H_0) = H_m$ by induction on the definition of $(H_n)_{n=0}^m$. It then turns out $H_0 = H^m(H_0)$. Thus, for all natural number i , $H^{im}(H_0) = H_0$. Moreover, because it is a cycle, we can be more general:

$$\forall i, j \in \mathbf{N}, H^i(H_j) = H^{i \bmod m}(H_{j \bmod m}) = H_{(i+j) \bmod m} \quad (1)$$

There's also a little more piece of information; since m is the smallest number for which we have a cycle,

$$\forall i \neq j \bmod m, H_i \neq H_j \quad (2)$$

Note how similar the structure is to Fermat's little theorem. This discovery is amazing, but I don't immediately see how Equation (1) helps me find a collision faster. On the other hand, the more trivial Equation (2) helps me, only a little. It essentially provides m distinct pairs of preimages and hash values, i.e., $(H_i, H_{i+1})_{i=0}^{m-1}$. So I benefit from here doing m less calculations.

But I fail to see how to use Equation (1) to help. I asked a question here <https://crypto.stackexchange.com/questions/115058/how-can-having-a-cycle-help-finding-a-hash-collision> which may find an answer later.

Another possible meaning I later read Section 6.4.3 of [Katz, Lindell, 3rd ed.], which seems to be more of the meaning of the questions. But this assumes that the length of the cycle equals exactly the cardinality of the range of H , which isn't explicitly mentioned in the question.

Well, anyway, so this possible scenario is essential the previous one plus this additional information:

$$m = \text{size of the range of } H \quad (3)$$

How does this seemingly unimportant information help? Well, it actually guarantees that every hash value falls into the cycle, which is very important. Under this strong guarantee, we don't have to store all experienced (preimage, hash) pair to succeed.

[Katz, Lindell] gives this even division scheme: partition the whole range, i.e., the cycle, into $\sqrt{m}$ segments, each of length $\sqrt{m}$. Here's the clever part: one only stores the start and end of each segment, i.e., $(H_{i\sqrt{m}}, H_{(i+1)\sqrt{m}})$. Thus, we only have to store $2\sqrt{m}$ pairs in worst case (actually, now, in all cases), whereas previously we needed to store m pairs.

The algorithm runs as follows: given any hash value y , we can find a preimage of it inside the range of H , because the cycle covers the whole range, y must fall into one of the segments.

1. If y is one of the end points $H_{(i+1)\sqrt{m}}$, then doing $\sqrt{m} - 1$ hashes from the start point gives a preimage of y .
2. If otherwise y is inside a segment, then applying H less than $\sqrt{m}$ times is guaranteed to reach a endpoint. Then, we find its startpoint, and start to do $\sqrt{m} - 1$ — the number of H on y to reach the endpoint hashes.

This gives an algorithm to find a preimage inside the range, for any hash value. Thus, to turn it into a collision, one simply picks any value out of the range, calculate its hash, and use the algorithm to find another preimage that's inside the range.

To store even less bits, reduce the number of segments, which will in turn increase the number of hashes we need to do. It is possible to store less than $\sqrt{m}$ bits while being able to still do it faster than brute force.